

Security Linter Report

Filename	case10_dynamic_columns.py
Timestamp	2026-06-05 04:29:12 UTC
Lines of Code (LOC)	25
Verdict	VULNERABLE

ID	Severity	Sink	Source Variable	Source Type	Line
CVE-SL-002	CRITICAL	cursor.execute	sql	STDIN	33
CVE-SL-003	CRITICAL	cursor.execute	table_name	STDIN	34

CRITICAL	2
WARNING	0
INFO	0
TOTAL	2

CVE-SL-002 — cursor.execute**CRITICAL**

CWE	CWE-89
Sink	cursor.execute
Sink Line	33
Sink Column	0
Sink Expression	cursor.execute(sql)
Argument	sql
Source Variable	sql
Source Type	STDIN
Propagation Mechanism	direct assignment from stdin

Taint Path Trace:

Step	Role	Line	Expression
1	SOURCE	10	sql = "SELECT " + column_name + " FROM " + table_name + " WH...
2	SINK	33	cursor.execute(sql)

Remediation:

Use a parameterized query and validate the input type. If the value must be an integer, cast it with int():

```
user_id = int(input("ID: "))
query = "SELECT * FROM users WHERE id = ?"
cursor.execute(query, (user_id,))
```

CVE-SL-003 — cursor.execute**CRITICAL**

CWE	CWE-89
Sink	cursor.execute
Sink Line	34
Sink Column	0
Sink Expression	cursor.execute(base_sql)
Argument	base_sql
Source Variable	table_name
Source Type	STDIN
Propagation Mechanism	direct assignment from stdin

Taint Path Trace:

Step	Role	Line	Expression
1	SOURCE	6	table_name = input("Enter table name: ")
2	ASSIGN	18	base_sql += " AND category = " + category + ""
3	SINK	34	cursor.execute(base_sql)

Remediation:

Use a parameterized query and validate the input type. If the value must be an integer, cast it with int():

```
user_id = int(input("ID: "))
query = "SELECT * FROM users WHERE id = ?"
cursor.execute(query, (user_id,))
```

No safe sink paths confirmed.

No sanitization functions detected.

Vulnerabilities	2 CRITICAL 0 WARNING 0 INFO
Safe Sinks	0
Analysis Time	41.2 ms
Engine Version	1.0.0
Final Verdict	VULNERABLE